



EYE-GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING APPROACH

A PROJECT REPORT

Submitted by

DHANARAJ R	621322104019
GOWTHAM S	621322104029
NITHISH S J	621322104061

*in partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

**KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)**

ANNA UNIVERSITY :: CHENNAI 600 025

APRIL 2026



KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)

Tholurpatti (Po), Thottiam (Tk), Trichy (Dt) – 621 215

COLLEGE VISION & MISSION STATEMENT

VISION

“To become an Internationally Renowned Institution in Technical Education, Research and Development by Transforming the Students into Competent Professionals with Leadership Skills and Ethical Values.”

MISSION

- Providing the Best Resources and Infrastructure.
- Creating Learner centric Environment and continuous Learning.
- Promoting Effective Links with Intellectuals and Industries.
- Enriching Employability and Entrepreneurial Skills.
- Adapting to Changes for Sustainable Development.

COMPUTER SCIENCE AND ENGINEERING

VISION & MISSION STATEMENT

VISION

To produce competent software professionals, academicians, researchers and entrepreneurs with moral values through quality education in the field of Computer Science and Engineering.

MISSION

- Enrich the students' knowledge and computing skills through innovative teaching-learning process with state-of-art- infrastructure facilities.
- Endeavour the students to become an entrepreneur and employable through adequate industry institute interaction.
- Inculcating leadership skills, professional communication skills with moral and ethical values to serve the society and focus on students' overall development

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

PEO I: Graduates shall be professionals with expertise in the fields of Software Engineering, Networking, Data Mining and Cloud computing and shall undertake Software Development, Teaching and Research.

PEO II: Graduates will analyze problems, design solutions and develop programs with sound Domain Knowledge.

PEO III: Graduates shall have professional ethics, team spirit, life-long learning, good oral and written communication skills and adopt corporate culture, core values and leadership skills.

PROGRAM SPECIFIC OUTCOMES (PSO's)

PSO1: Professional skills: Students shall understand, analyses and develop computer applications in the field of Data Mining/Analytics, Cloud Computing, Networking etc., to meet the requirements of industry and society.

PSO2: Competency: Students shall qualify at the State, National and International level competitive examination for employment, higher studies and research.

PROGRAM OUTCOMES (PO's)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life Long learning:** Recognize the need for, and have the preparation and ability to engage independent and life-long learning in the broadest context of technological change.

**KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)**

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **“EYE-GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING APPROACH”** is the bonafide work of **DHANARAJ R (621322104019), GOWTHAM S (621322104029), NITHISH S J (621322104061)** who carried out the project work under my supervision.

SIGNATURE

Dr. C. Saravanabhavan, M.Tech., Ph.D.

HEAD OF THE DEPARTMENT

Department of Computer Science and
Engineering

Kongunadu College of Engineering
and Technology, Thottiam, Trichy.

SIGNATURE

Dr. A. Rajaprabu, M.E., Ph.D.

SUPERVISOR

Assistant Professor,

Department of Computer Science and
Engineering

Kongunadu College of Engineering
and Technology, Thottiam, Trichy.

Submitted for the Project Viva-Voce examination held on_____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We wish to express our sincere thanks to our beloved, respectful and Honorable Chairman **Dr.PSK.R.PERIASWAMY** for providing immense facilities in our institution.

We proudly render our thanks to our Respected Principal **Dr.R.ASOKAN, M.S.,M.Tech., Ph.D.**, for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express our gratitude to our Respectful Dean(R&D), **Dr.J.YOGAPRIYA,M.E.,Ph.D.**, Dean (Academics & IQAC Co-Ordinator) **Dr.J.PREETHA, M.E.,Ph.D.**, who are the continual source of inspiration to think innovatively to complete the project.

We proudly render our immense gratitude to our Head of the Department of Computer Science and Engineering **Dr.C.SARAVANABHAVAN, M.Tech, Ph.D.**, for his effective leadership, encouragement and guidance in the project.

We highly indebted to provide our heartfelt thanks to our respectful Supervisor., **Dr. A. RAJAPRABU, M.E., Ph.D** for his valuable suggestion during execution of our project work.

We highly indebted to provide our heartfelt thanks to our respectful project coordinator **Mr. R. MADANACHITRAN, M.E.**, for his valuable ideas, constant encouragement and supportive guidance throughout the project.

We wish to extend our sincere thanks to all teaching and non-teaching staffs of Computer Science and Engineering department for their valuable suggestions, cooperation and encouragement on successful completion of this project.

We wish to acknowledge the help received from various departments and various individuals during the preparation and editing stages of the Manu script.

ABSTRACT

Individuals with locomotor disabilities or paralysis often struggle with independent mobility due to their inability to operate traditional wheelchairs that require manual or joystick control. This affects their autonomy and quality of life, making them dependent on caregivers for daily movement. To address this challenge, this project proposes an intelligent eye-tracking based smart wheelchair control concept that enables users to navigate and interact using only their eye movements. In the initial phase, a software-based system is developed, where real-time eye movements are captured using a webcam and processed with OpenCV and Python to detect gaze direction and blinks. These outputs are classified and mapped into commands such as forward, backward, left, right, stop and PDF scrolling. As a future enhancement, these commands will be integrated with Raspberry Pi and wireless communication for real wheelchair control. This AI-driven solution aims to provide a hands-free, real-time assistive system that enhances independence and safety for physically disabled individuals.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	vii
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	1
	1.1 OVERVIEW	1
	1.2 OBJECTIVE	2
	1.3 INTRODUCTION TO ML AND DL	3
	1.4 SDG INTEGRATION IN PROJECT DEVELOPMENT	4
2	LITERATURE REVIEW	6
3	SYSTEM ANALYSIS	11
	3.1 EXISTING SYSTEM	11
	3.1.1 Disadvantages	12
	3.2 PROBLEM STATEMENT	13
	3.3 PROPOSED SYSTEM	13
	3.3.1 Advantages	16
4	SYSTEM REQUIREMENTS	17
	4.1 SYSTEM SPECIFICATIONS	17
	4.1.1 Hardware Requirements	17

	4.1.2 Software Requirements	17
5	SYSTEM DESIGN	18
	5.1 SYSTEM ARCHITECTURE	18
	5.2 DATA FLOW DIAGRAM	20
6	SYSTEM IMPLEMENTATION	24
	6.1 MODULES	24
	6.2 MODULES DESCRIPTION	24
	6.2.1 Face Detection	24
	6.2.2 Eye Region Detection	24
	6.2.3 Iris Region Extraction	25
	6.2.4 Preprocessing	25
	6.2.5 Feature Extraction	25
	6.2.6 ML / DL Classification	26
	6.2.7 Decision Mapping & Command Generation	26
	6.3 IMPLEMENTATION	27
7	SYSTEM TESTING	30
	7.1 TESTING OBJECTIVES	30
	7.2 TYPES OF TESTING	30
	7.2.1 System Testing	30
	7.2.2 Unit Testing	30
	7.2.3 Integration Testing	30

	7.2.4 Validation Testing	30
	7.2.5 Functional Testing	31
	7.3 TEST CASES	31
8	RESULT AND DISCUSSION	34
	8.1 PERFORMANCE ANALYSIS	34
9	CONCLUSION AND FUTURE ENHANCEMENT	35
	9.1 CONCLUSION	35
	9.2 FUTURE ENHANCEMENT	35
	APPENDICES	36
	REFERENCES	50
	PUBLICATION	54

LIST OF FIGURES

FIG.NO	NAME OF THE FIGURE	PAGE NO
5.1	System Architecture	18
5.2	Data Flow Diagram Level 0	20
5.3	Data Flow Diagram Level 1	21
5.4	Data Flow Diagram Level 2	22
6.2	Face Detection with Haar Features	28
7.1	Confusion Matrix	32
7.2	Classification Report	33
1	Forward Gaze Wheelchair Movement	46
2	Left Gaze Wheelchair Control	46
3	Right Gaze Wheelchair Control	47
4	Eye Blink Wheelchair Stop	47
5	Upward Gaze Document Scroll	48
6	Downward Gaze Document Scroll	48
7	Center Gaze Document Pause	49
8	No Eye Detected – System Stop	49

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CNN	Convolutional Neural Network
CV	Computer Vision
DL	Deep Learning
EAR	Eye Aspect Ratio
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GUI	Graphical User Interface
HCI	Human Computer Interaction
IoT	Internet of Things
ML	Machine Learning
PDF	Portable Document Format
RGB	Red Green Blue
ROI	Region of Interest
SVM	Support Vector Machine
TN	True Negative
TP	True Positive

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The growing importance of intelligent assistive systems is critical to enhancing physical disabilities quality of life. Human-computer interaction fills the important function of allowing users to interact with and control devices. However, traditional interaction mechanisms are based on physical inputs like hand movement or external controllers which may not be accessible for individuals with limited mobility. This leads us to the conclusion that there continues to be a demand for an intuitive, efficient hands-free human interface solution that can comprehensively support communication with a computer through natural human signals.

Such interaction systems require accurate detection and interpretation of all important visual cues and eye movements in particular. Traditional techniques are based on manual inspection or simple image processing algorithms that typically remain inefficient, less accurate, and sensitive to environmental conditions. Eye movement detection has become a potential solution with advances in computer vision and deep learning. The analysis of gaze direction and blinking pattern can be performed in real time using facial landmark detection, iris tracking, and machine learning-based classification methods. Those innovations allow us to interact quicker, more accurately and work less.

Accurate prediction and interpretation of eye movement patterns is essential to develop real-time systems in a reliable manner. Conventional approaches are restricted due to factors such as noise, differences in illumination and non-robustness. This helps the system obtain more accurate and stable performance by combining deep learning model with

feature extraction and preprocessing processing techniques. Robust algorithms for accurate gaze region detection, meaningful features extraction and intent classification. This method improves the overall performance of a system by enabling a scalable and real-time solution for human-computer interaction which in turn increases usability and responsiveness of the system.

Data exchange and knowledge sharing promoted the development of modern intelligent frameworks that leverage computer vision techniques alongside machine learning methods for efficient, real-time processing with strong results. Such systems feature sophisticated patterns extraction and classification models as well engineered processing pipelines for the purpose of robust handling with difficult scenarios. Frameworks like these, developed for practical uses offer a way to overcome this by providing an optimized and user-centric system that is dependent on enhancing accessibility and ease of use rather than interaction with physical objects in the real world which results in a better user experience.

1.2 OBJECTIVE

The Objectives of the Proposed Eye-Based Intelligent Interaction Framework are,

- Permitting hands-free control with eye movement detection so that it is easy to interact.
- Computer vision methods for real-time gaze detection.
- Command selection and emergency control through blink detection systems.
- Leverage advanced feature extraction techniques and predictive models to improve detection accuracy across diverse conditions.
- Depending upon the algorithms, optimize for real-time processing to keep the system fluid and responsive. Make the system more robust

and try to overcome challenges connected with lighting variations, noise, and motion of head.

- This means less dependence on physical interactions, thus promoting better accessibility and independence.
- Reduce dependency on physical input methods, improving accessibility and user independence.
- Provide a scalable and low-cost solution using standard cameras and software-based implementation.

1.3 INTRODUCTION TO ML AND DL

Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn patterns from data and make predictions without explicit programming. In this project, ML is used to classify eye movements such as left, right, forward, and blink based on extracted features. Techniques like Haar Cascade are applied for face detection, while MediaPipe is used for accurate facial landmark and eye region detection. These methods help in efficient preprocessing and feature extraction.

Deep Learning (DL) is an advanced form of ML that uses neural networks to automatically learn complex patterns from image data. In this system, Convolutional Neural Networks (CNN) are used for classifying eye gaze directions and blink states. DL improves accuracy by learning spatial features of the eye region without manual intervention.

The integration of ML techniques (Haar Cascade, MediaPipe) and DL models (CNN) ensures accurate detection, reliable classification, and real-time performance, making the system effective for assistive applications and human–computer interaction.

1.4 SDG INTEGRATION IN PROJECT DEVELOPMENT

SDG 3: Good Health and Well-being

The ReLU activation function introduces non-linearity, enabling the model to learn complex patterns. CNNs use forward and backpropagation to adjust weights and minimize errors. They perform well due to automatic learning, adaptability, and high accuracy, making them suitable for real-time applications.

SDG 4: Quality Education

Encourages inclusive education by supporting hands-free interaction, making it easier for individuals with physical disabilities to navigate documents and access digital content. With this system, users can control actions using eye movements, allowing them to scroll and interact with learning materials without needing physical input.

SDG 9: Industry, Innovation and Infrastructure

Drives innovation by combining artificial intelligence, computer vision, and embedded systems to create advanced solutions. In this case, AI-powered eye tracking integrated with OpenCV and embedded platforms makes it possible to control a smart wheelchair in real time.

SDG 10: Reduced Inequalities

Supports inclusivity by offering assistive technology that helps physically disabled individuals gain equal access to mobility and everyday activities. Through eye-tracking technology, differently-abled individuals are given more independence and equal opportunities to move and interact with their environment.

CHAPTER 2

LITERATURE REVIEW

[1] TITLE: Humanoid-Human Sit-to-Stand-to-Sit Assistance (2025)

AUTHOR: Chaki Tomohiro, Kawakami Tomohiro, Lefèvre Hugo, Tanguy Arnaud.

A novel framework for assisting human sit-to-stand and stand-to-sit movements using humanoid robots is developed by analyzing human dynamic balance. The method computes assistive forces based on balance criteria such as the Divergent Component of Motion (DCM) and Virtual Repellent Point (VRP), enabling accurate support during motion. Human posture, motion, and external forces are continuously monitored using motion capture systems and force sensors to estimate balance conditions in real time. The humanoid applies calculated forces at appropriate body locations to stabilize the user while maintaining its own balance. The system integrates dynamic models, multi-contact force distribution, and feedback control to ensure smooth and safe interaction. Experimental results with humanoid robots demonstrate effective assistance under simulated frailty conditions, though performance depends on sensor accuracy and modeling assumptions. The approach highlights the potential of humanoids in assistive robotics applications.

Algorithm : DCM-VRP based Control & Multi-body Dynamics

Merit : Real-time adaptive assistance based on human balance.

Demerits : Requires accurate sensors and complex modeling.

[2] TITLE: E-Gaze: Gaze Estimation with Event Camera (2024)

AUTHOR: Chang Muya, Li Nealson, Arijit Raychowdhury.

A real-time gaze estimation system using event cameras is developed to achieve high accuracy and low latency in near-eye tracking applications. The approach utilizes event-based data, which captures only changes in pixel intensity, enabling efficient processing of fast eye movements. Eye features are extracted by analyzing spatial, temporal, and polarity distributions of events, and pupil regions are represented as ellipses. A sequence of these features is fed into a recurrent neural network (RNN) with Long Short-Term Memory (LSTM) layers to model temporal dynamics of eye motion. A novel angular loss function improves prediction accuracy by minimizing angular error rather than pixel distance. The system operates without additional hardware such as infrared LEDs and achieves high performance with an accuracy of 0.46° and latency below 1 ms. It demonstrates robustness across different users and lighting conditions, though complexity in event processing remains a challenge.

Algorithm : Event-based Feature Extraction + RNN (LSTM)

Merit : High accuracy with ultra-low latency using minimal hardware.

Demerit : Complex processing of event-based data.

[3] TITLE: Approach to Quantify Eye Movements to Augment Stroke Diagnosis with a Non-Calibrated Eye-Tracker (2023)

AUTHOR: Aldridge Chad M, Hassan Mohamed Abul, Zhuang Yan.

An approach is developed to analyze eye movements for stroke diagnosis using a non-calibrated eye-tracking system. The method focuses on measuring eye conjugacy, which represents the coordination between left and right eye movements. Instead of relying on calibration, the system evaluates correlation patterns using statistical measures such as Spearman's rank correlation. Various eye movement tests including Dot, H-pattern, head orientation, and optokinetic nystagmus are used to capture different ocular motor functions. Results show that calibration does not significantly affect the correlation between eye movements in healthy individuals, while patients exhibit lower correlation values, indicating abnormalities. The system successfully distinguishes normal and pathological eye behavior using non-calibrated data. This approach improves feasibility in clinical environments where patients cannot complete calibration procedures, especially in neurological disorders like stroke.

Algorithm : Eye Conjugacy Analysis using Spearman Correlation

Merit : Works without calibration; suitable for real-time clinical use.

Demerit : Requires larger datasets for improved accuracy and validation.

[4] TITLE: Multiview Multitask Gaze Estimation with Deep Convolutional Neural Networks (2019)

AUTHOR: Duan Lixin, Hu Lina, Lian Dongze, Luo Weixin, Xu Yanyu.

A deep learning-based gaze estimation approach is proposed using multi-view and multitask convolutional neural networks (CNNs) to improve accuracy and robustness. The method processes facial images captured from multiple viewpoints and learns shared representations for gaze direction estimation. It integrates multitask learning by jointly predicting gaze angles along with auxiliary features such as head pose, enhancing model performance. The CNN automatically extracts discriminative features, reducing the need for manual feature engineering. Data from different views improves generalization and adaptability under varying conditions. Experimental results demonstrate high accuracy and robustness compared to traditional gaze estimation methods. The model supports real-time applications and performs effectively across different users and environments. However, it requires large training datasets and high computational resources for optimal performance.

Algorithm : Multiview CNN + Multitask Learning

Merit : High accuracy with strong generalization across different viewpoints.

Demerit : Requires large datasets and high computational cost.

[5] TITLE: Eye Gaze Tracking With a Web Camera in a Desktop Environment (2015)

AUTHOR: Qinmu Peng, Yiu-ming Cheung.

A low-cost eye gaze tracking system is developed using a standard web camera for desktop environments. The approach focuses on visible imaging instead of infrared systems, making it more practical and affordable. The method detects facial features using an active shape model and extracts eye regions under varying lighting conditions. The iris center is identified by combining intensity energy and edge strength, while the inner eye corner is detected using curvature-based techniques. A gaze vector is formed from these features and mapped to screen coordinates through a polynomial calibration model. To improve accuracy, head movement is compensated using a sinusoidal head model and an adaptive weighted POSIT algorithm. Experimental evaluation shows high accuracy under different lighting conditions, achieving around 1.28° error without head movement.

Algorithm : AWPOSIT (Adaptive Weighted POSIT) with Eye
Feature Detection

Merit : Low-cost, robust to lighting, no special hardware

Demerit : Sensitive to extreme head pose and partial eye occlusion.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

According to the research, there is a standard method on eye-movement detection and interaction which utilizes traditional computer vision methods and machine learning techniques such as Haar Cascade, facial landmark, or simple gaze estimation. These systems track facial features using webcam data, follow eye areas as a means of interaction.

Because of the fast-processing time it uses Haar Cascade classifiers for face and eye detection in those implementations. While Haar Cascade for the detection is quick enough and can process video in real-time but they do not fare well with different head poses and different lighting conditions. A bolted-on piece of intermediate functionality on this is the stability. Generally, missing key point detection around eye is commonly seen in many facial landmark detections therefore the eye region localization. Well these most successful methods are dependent highly upon the geometric constraints and rules, they can possibly lead to inaccurate results in ever changing dynamic environments.

The current systems are mainly based on EAR-based blink detection for identifying events such as eye closure. They then measure a ratio of eye landmarks distances and identify blinks. Although EAR is easy to compute and computationally light, it can be sensitive levels of frame rate, illumination, and user non stationarity, therefore it needs tight threshold tuning.

Traditional techniques for gaze estimation are geometric and appearance-based methods recognizing predetermined eye direction. But those techniques have problems with accuracy, e.g. on eye movements and occlusions. This makes most of the existing systems based on static

features and hand-crafted features during inference almost impractical for real-time applications, which forcefully caps their performance.

Methods for tracking in existing systems are fairly primitive and do not take advantage of advanced temporal modeling or deep learning based tracking. So, due to noise, motion blur or partial occlusion, it is difficult to ensure the continuous and full cross-frame homogeneity of gaze detection.

Existing implementations (which seem to be the standard) would need improvements in terms of accuracy, sensitivity for environmental conditions and scalability. The essence of these bindings constrained and require advanced methods learned around deep learning alongside solid features extraction coupled with retuned composition to assure system real-time for improved robustness.

3.1.1 Disadvantages

- Lighting sensitivity - Performance suffers in darkness, shadows or when lighting conditions change abruptly, with implications for accurate eye detection and tracking.
- Head movement limitations - When the user moves their head, consistent tracking is difficult to maintain leading to incorrect gaze estimation.
- Threshold dependency - Blink detection methods such as EAR require exact threshold tuning and may differ between users and settings.

3.2 PROBLEM STATEMENT

Current human–computer interaction methods are not suitable for individuals with limited mobility, as they rely heavily on physical input devices such as keyboards, joysticks, or touch interfaces. Existing eye-tracking systems often suffer from low accuracy due to variations in lighting conditions, head movements, and partial occlusions, along with limitations in real-time responsiveness. These challenges reduce the reliability and usability of such systems in practical scenarios.

This work addresses these issues by developing a vision-based system that integrates face detection, eye tracking, gaze estimation, and blink recognition using optimized machine learning and deep learning techniques. The system employs robust preprocessing methods to handle illumination changes and noise, ensuring consistent performance across different environments. Advanced feature extraction techniques are used to accurately capture iris position and eye state, enabling precise classification of gaze directions and blink events.

This work addresses these issues by developing a vision-based system that integrates face detection, eye tracking, gaze estimation, and blink recognition using optimized machine learning and deep learning techniques. The system employs robust preprocessing methods to handle illumination changes and noise, ensuring consistent performance across different environments. Advanced feature extraction techniques are used to accurately capture iris position and eye state, enabling precise classification of gaze directions and blink events.

3.3 PROPOSED SYSTEM

The proposed system utilizes the Haar Cascade Algorithm for fast face detection combined with advanced eye-tracking and machine learning techniques to achieve accurate real-time interaction. Haar

Cascade detects the facial region efficiently using feature-based classifiers with low computational cost, making it suitable for real-time webcam input. After detecting the face, the system extracts the eye region using facial landmark detection and Region of Interest (ROI) methods.

Preprocessing techniques such as grayscale conversion, normalization, and noise reduction are applied to improve robustness under varying lighting conditions. The processed eye region is then used for feature extraction, including iris position, blink patterns, and eye movement tracking. These features are analyzed using machine learning and deep learning models such as CNN for accurate gaze classification. Finally, the system maps the detected eye movements into control commands for wheelchair navigation and document scrolling, enabling a completely hands-free interaction system.

The invention in particular solves problems with eye-based interaction systems:

1. **Face Detection and Localization**

The Haar Cascade algorithm is used to detect faces from real-time video frames by scanning the image at multiple scales using Haar-like features. It employs a cascade of classifiers that quickly eliminates non-face regions and focuses only on relevant areas. This improves detection speed and ensures that the system can operate efficiently in real-time. The method is robust for frontal face detection and works well under moderate variations in lighting and background conditions.

2. **Eye Region Extraction**

Once the face is detected, the system isolates the eye region using facial landmark detection techniques such as MediaPipe. These landmarks help accurately locate the position of the eyes within the detected face. The Region of Interest (ROI) is then extracted to focus only on the eye

area, reducing unnecessary computations. This step improves precision by ensuring that only relevant visual information is processed for further analysis.

3. **Preprocessing and Enhancement**

The extracted eye region undergoes preprocessing to improve image quality and consistency. Grayscale conversion reduces computational complexity by eliminating color information. Noise reduction techniques help remove unwanted distortions, while normalization ensures uniform pixel intensity across frames. These steps enhance the visibility of important features such as iris boundaries and eyelid movement, making the system more reliable under different lighting conditions.

4. **Feature Extraction**

In this stage, important features such as iris position, eye aspect ratio (EAR), and blink patterns are extracted from the processed image. The iris position helps determine gaze direction, while EAR is used to detect eye closure and blinking. These features are crucial for understanding user intent and are extracted continuously from video frames to enable real-time analysis.

5. **Gaze and Blink Classification**

Machine Learning and Deep Learning models, particularly Convolutional Neural Networks (CNN), are used to classify eye movements into categories such as left, right, forward, and close. The model learns spatial patterns from eye images and accurately predicts gaze direction. This classification process ensures that even slight variations in eye movement are correctly interpreted, improving system accuracy and reliability.

6. **Real-Time Processing**

The system is designed for real-time performance with minimal latency. Efficient algorithms and optimized processing pipelines ensure that each video frame is analyzed quickly. This allows the system to respond instantly to user eye movements, providing smooth and continuous interaction. Real-time processing is essential for applications like wheelchair control, where immediate response is critical for user safety.

7. **Command Mapping and Output Control**

The classified eye movements are converted into control commands that drive system outputs. For example, left gaze results in left movement, right gaze triggers right movement, forward gaze enables forward motion, and eye closure stops the system. Similarly, upward and downward gaze controls document scrolling, while center gaze pauses it. This mapping ensures intuitive and natural interaction, allowing users to control the system without physical input.

3.3.1 Advantages

- **Rapid Face Detection** - The Haar Cascade classifier exploits simple features-based classification, and thus it is perfect for use in real-time.
- **Real-Time Processing Ability** – Due to the optimized architecture, processing can be done on a near real-time basis as opposed to CNN which is relatively slower.
- **Live Performance** - Process faster and respond so user experience becomes fast and eco-friendly without delay.
- **No Physically-Driven control** – The users can use eye movements and blinks to operate the systems without using any devices.
- **Increased Accessibility** – Higher user-friendliness and autonomy for those who find it challenging to move around.
- **Economic Deployment** – Leverages regular webcams and software with no costly hardware demands.
- **Feature Detection** - Haar-like features work great in finding the facial areas with small assumptions.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 SYSTEM SPECIFICATIONS

4.1.1 Hardware Requirements

- Devices : Computer / Laptop (Windows 11)
- Processor : Intel i3 or above
- Ram size : Minimum 4GB
- Camera : HD Webcam

4.1.2 Software Requirements

- Operating System : Windows 7
- Language : Python
- Model : MediaPipe / Haar CasCade
- Libraries : OpenCV , NumPy , Tensorflow , Keras
- IDE : Visual Studio Code

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

Fig. No: 5.1 shows the system design where real-time webcam video is processed to detect eye movements and control wheelchair navigation and document scrolling using machine learning.

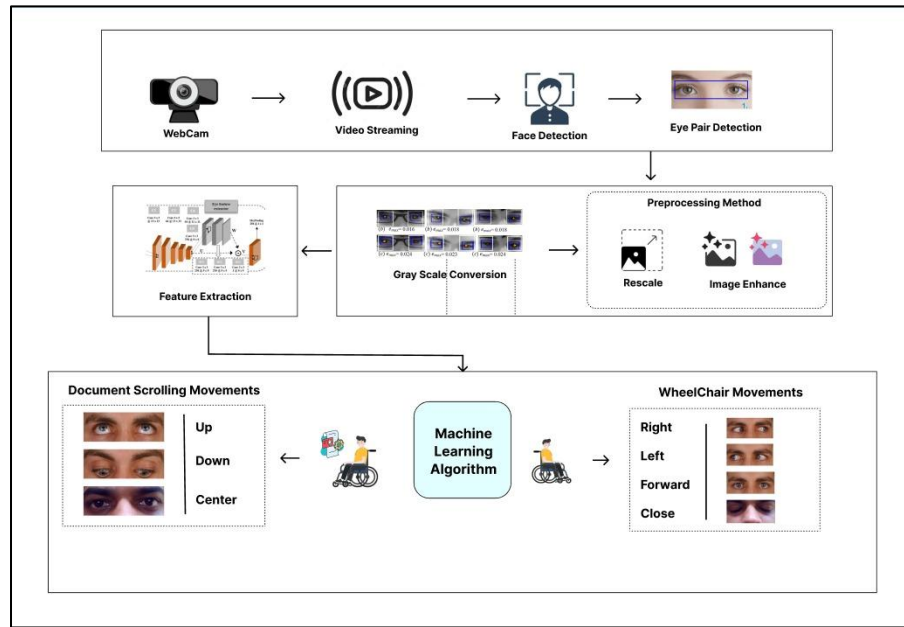


Fig. No. 5.1: System Architecture

The diagram illustrates an eye-based interaction pipeline with two stages: Eye Detection & Processing and Command Generation & Output Control.

1. Image Acquisition & Face Detection:

- First, it captures real-time video from the webcam and processes each consecutive frame.
- Efficiently detecting the face region by using the Haar Cascade algorithm which identifies haar like features like edges, intensity differences etc.
- Faster cascade classifier, which uses multiple stages of classifiers to reject most regions quickly and leaves all qualified regions for further

processing.

- This prevents unnecessary computation of eyes in other places along with ensuring that the user's face is accurately localized.

2. Eye Detection & Region Localization:

- Once the face is detected, we extract the eye region based on predefined coordinates for facial landmarks or landmark-based technique.
- The preprocessing based on eye pair detection helps the face detector process about only the area it requires, thus improving speed, as well as efficiency.
- This stage allows correctly recognizing both eyes even when the head position is slightly different. Localization is done with careful consideration, as these details will be crucial for accurate gaze tracking and blink detection in subsequent steps.

3. Preprocessing & Image Enhancement:

- The eye region which has been extracted is pre-processed to enhance the quality and consistency of input images.
- Grayscale conversion, noise reduction, normalization and resizing of the image.
- Contrast and brightness normalizing techniques help deal with fluctuating lighting. It removes the noise and enhances the eye features necessary for model prediction.

4. Feature Extraction & Classification:

- Processed images are used to extract important features like iris position, eyelid movement and shape.
- It is also used to infer gaze direction (left, right, forward) and detect blink patterns. A trained machine peeks through these features and identifies whether the user is looking to buy, find out more about a product, or merely browse.

- This is a key step to ensure real-time and reliable understanding of eye movements in the right way.

5. Command Generation & System Control:

- Control commands for system operation are generated from the classified outputs.
- Eye movements give rise to commands including left, right, forward and stop.
- Detecting sudden eye blinks can signal to pick or for an emergency.
- Additional features like scrolling through documents with eye direction are also available. This module makes real-time response seamless, providing an intuitive and hands-free experience.

5.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) visually represents a system's functionality, showing how input data flows through processing stages to produce output.

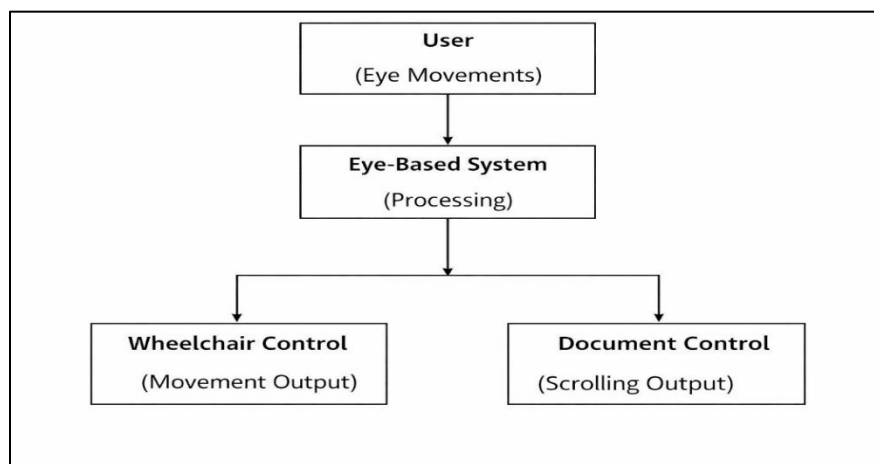


Fig. No. 5.2: Data Flow Diagram Level 0

The process in the Level 0 DFD's system is simplified to a single processing unit that takes eye movements and translates them into commands. Eye-Based System: The Eye-Based System used computer vision and machine learning techniques to process live stream image input

as the User provides eye movement data. The processed data is simply separated into two outputs: Wheelchair Control, which provides navigation commands and Document Control, to allow scrolling or interaction. The higher level of this describes the complete input-output without going into depth about what is happening on an internal level.

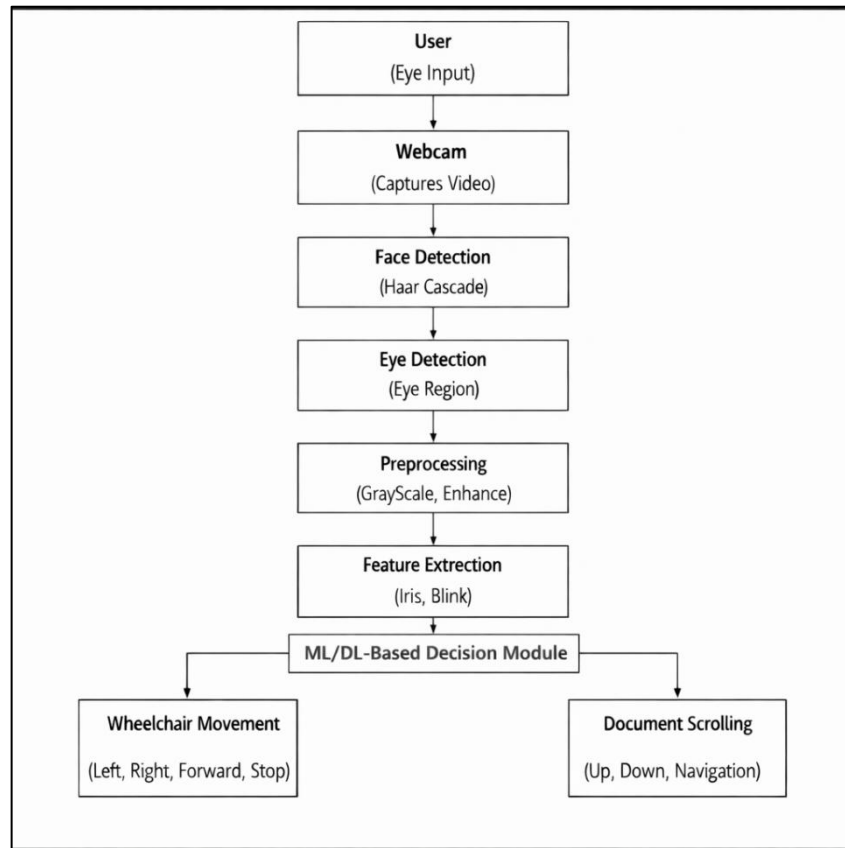


Fig. No. 5.3: Data Flow Diagram Level 1

The Level 1 DFD represents the system's detailed functionality, divided into major modules. Real-time video frames are captured from the webcam as user input. These frames undergo Face Detection (Haar Cascade) to locate the face, followed by Eye Detection to extract the eye region. The extracted data is preprocessed using grayscale conversion and enhancement for improved quality. In the Feature Extraction stage, key parameters such as iris position and blink patterns are identified. These

features are then fed into a Machine Learning model for gaze direction and blink classification. Finally, the classified output is converted into control signals for Wheelchair Movement and Document Scrolling, enabling real-time system interaction.

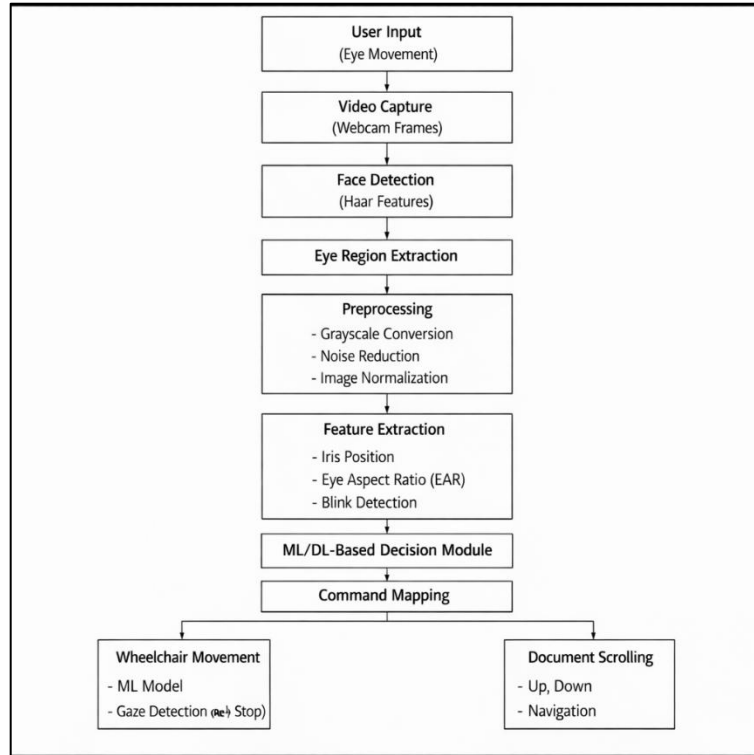


Fig. No. 5.4: Data Flow Diagram Level 2

This flows into a Level 2 DFD which breaks down the internal processing stages leading to useful control outputs, showing how raw eye movement data gets transformed through multiple computation steps along the way. It starts from User Input, the eye movements (blinking and movement) are captured in real-time using a webcam.

The Video Capture module used there for creating a video from continuous frames and our face Image is processed with Face Detection (Haar Cascade) to find the portion of face in input frame.

Eye Region Extraction is then performed, which isolates the eye region before taking any decisions on it. Next, the extracted eye region is Pre-processed by Grayscale conversion, Noise Reduction, and normalization to increase the image quality of varying conditions. Subsequently, by performing Feature Extraction to grab few necessary parameters like iris position, eyelid movement and Eye Aspect Ratio (EAR) which assist in detecting gaze direction and blink patterns.

These features are then fed into a Classification module as input, where a machine learning model provides predictions for the gaze direction (left or right or forward) and blink events. Output is mapped into specific control response in Command Mapping stage.

Finally, the Output Module provides commands for Wheelchair Control (movement and stop) and Document Scrolling (navigation) with such accuracy that they are directly performed in real-time without any hand usage.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 MODULES

The proposed model for the eye-based interaction system includes the following modules,

- FACE DETECTION
- EYE REGION DETECTION
- IRIS REGION EXTRACTION
- PREPROCESSING
- FEATURE EXTRACTION
- ML / DL CLASSIFICATION
- DECISION MAPPING & COMMAND GENERATION

6.2 MODULES DESCRIPTION

6.2.1 Face Detection

The face region is detected from the real-time video input using this module with the help of Haar Cascade algorithm. The algorithm functions by looking at the image on different scales checking for patterns that seem to be present in a face using Haar-like qualities (i.e. boundaries or intensity changes). The cascade classifier discards non-face areas in stages, which increases speed and improves efficiency. It helps in processing only the face area further which reduces computational load and better accuracy. It also allows for slight differences in orientation and distance from the camera.

6.2.2. Eye Region Detection

Once the face has been detected, this module will use region-of-interest (ROI) techniques or facial landmark detection to isolate the eye region. It even maintains its precision when the head shifts slightly,

isolating both left and right eyes. It makes it faster by limiting the area where you will be processing, which avoids unnecessary calculations. It guarantees consistent eye bounding which is critical for accurate gaze estimation and blink detection. It assists in keeping the stability by preventing fluctuations and swings between different slots of users.

6.2.3. Iris Region Extraction

Well, this module focuses on iris and pupil detection in eye region. It linearly separates the iris center and boundary points using image processing algorithms like thresholding, external contour detection. The iris moves across the frame and is used to determine gaze. Since even tiny movement differences correspond to different user intents, accurate iris extraction is key to good eye tracking. It famous this module which increases the sensitivity of your response to the system.

6.2.4. Preprocessing

This complete eye region is processed to achieve better image quality and consistency. This may involve converting images to grayscale to simplify data, filtering noise, normalizing for light variations and resizing for a consistent input size. The different preprocessing operations enhance feature clarity and maintain consistent performance checking under various atmospheric conditions. This preprocessing step needs to be done correctly which is of utmost importance to minimize errors in feature extraction and classification stages.

6.2.5. Feature Extraction

Key features like the iris positions, distance between the eyelids, and Eye Aspect Ratio (EAR) are extracted in this stage. EAR: The Eye Aspect Ratio (EAR) is used for blink event identification by measuring the ratio with respect to eye landmark points. Other characteristics such as eye contour or movement patterns are also taken into account. These features serve as input for the classification model and have a critical role

in system accuracy. Well-extracted features allow us to make sure the User Intent is extracted correctly.

6.2.6. ML / DL Classification

The features that have been extracted are then input into ML or DL models to classify the images. This model learns and predicts the gaze direction (left, right and forward), along with a blink event. Similar to the chart above, but shows similarities with users or environments instead of traits. It is responsible for converting raw feature data into meaningful outputs, allowing the system to make intelligent decisions and react in real-time.

6.2.7. Decision Mapping & Command Generation

Then machine learning or deep learning algorithms are applied to classify features extracted. The model detects gaze direction (left, right and straight) as well as blink events by examining data trends. Improvement of accuracy and adaptability across several users is achieved by training data. It converts the raw feature data into meaningful outputs to make intelligent decisions and respond in real time.

ALGORITHM

Step 1: Face and Eye Detection :

1. To process video frame-by-frame, use a webcam to capture real-time input.
2. Detecting face region using Haar Cascade based on extraction of Haar-like features.
3. Use MediaPipe Face Mesh to find facial landmarks.
4. Locate and extract accurate eye regions using landmark coordinates.

Step 2: Iris Tracking and Blink Detection :

5. Use MediaPipe Iris Tracking to get the position of the irises for gaze estimation.
6. Track iris motion and to decide if it is left, right, or straight.

7. Use Eye Aspect Ratio (EAR) or landmark-based methods for eye blink detection. Recognize blink patterns for commands and safety functions.

Step 3: Preprocessing and Feature Extraction :

8. Grayscale images - convert eye region images to grayscale images lower computational complexity. Instead, they're trained and engineered for things like noise reduction and normalization to enable images with consistent quality.
9. Discussion points in your description: iris position, eyelid distance, eye contours For blink detection accuracy, compute Eye Aspect Ratio (EAR).

Step 4: Classification (CNN Model)

10. Feed input cam features to a trained CNN model.
11. Gaze movement are classified based on left, right and forward.
12. Basic system for blink detection and classification. It enhance prediction accuracy using trained datasets and optimization of model.

Step 5: Command Generation and Output

13. Such as mapping classification outputs to control commands and pass them on based on some predefined logic.
14. These include commands for movement such as left, right, forward, and stop.
15. Enable actions that are specific to the document, such as scrolling and selection.
16. Maintain low latency for virtually instantaneous interaction.

6.3 IMPLEMENTATION

This eye-based interaction system is implemented using a pipeline architecture consisting of face detection, eye tracking with feature extraction based on affine transformations and real-time command generation. The system is composed of different modules that interact

between each other, starting from the video acquisition module in which real-time physical input is acquired via a webcam. So as one frame gets computed, it begins processing the next — ensuring quick and fluid interaction.

Haar Cascade algorithm

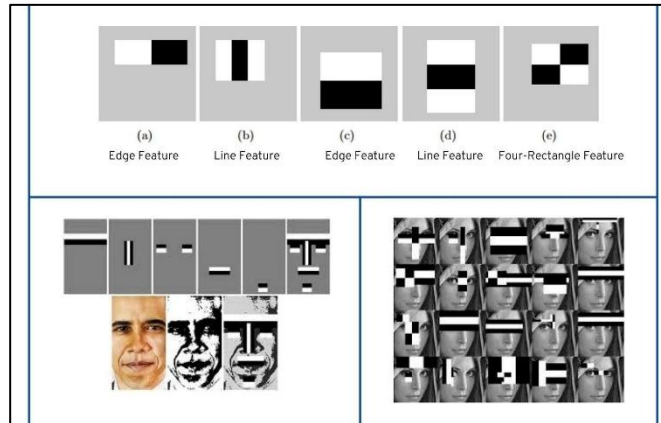


Fig No.6.1: Face Detection with Haar Features

The Haar Cascade algorithm is the face detection that occurs in Inter and the following sentence. The face region is efficiently characterized using Haar-like features and proceeds with cascade classifiers. For eye region detection, as soon as the face is detected, MediaPipe Face Mesh comes into play; it extracts facial landmarks in order to accurately localize the eye region. This guarantees recognition of each eye when the tilt of the head slightly shifts.

Segmented eye area goes through preprocessing steps that involve: gray scaling; noise reduction; normalization After that, we use MediaPipe Iris Tracking to find the iris position and track its motion from frame to frame. As an example, this is utilized in gaze direction detection left, right, forward etc. At the same time, eye closure events are detected using Ear Aspect Ratio (EAR) or landmark-based approaches to be used further in blinking detection.

Then, extracted relevant features like iris position, eyelid movement, and blink patterns to feed into a CNN-based classifier. The model is trained to predict user intent based on the patterns in these features. These can be: gaze directions and blink signals.

The decision mapping module handles converting the predicted outputs into control commands.

These include command for interaction with the system like control of navigation and scrolling through documents. Real-time response is achieved through frame-processing optimization and lamely latency.

The processed frames with features detected are displayed using OpenCV, and the outputs are updated continuously based on user input. Our implementation is fast, inexpensive, efficient and able to deliver in real time for practical applications.

Metrics like accuracy, responsiveness and detection consistency are used to assess the overall system performance. Robust and reliable testing under varying levels of lighting and user differences.

CHAPTER 7

SYSTEM TESTING

7.1 TESTING OBJECTIVES:

Testing is done to find bugs and verify that the system works appropriately. It ensures that each individual module work correctly and the system behaves consistently under different conditions.

7.2 TYPES OF TESTING

7.2.1 System Testing

System testing verifies the overall functionality of the system. It ensures all modules—face detection, eye tracking, feature extraction, classification, and command generation—work together correctly. Testing is performed under varying conditions like lighting changes and user movement to confirm the system meets its specifications.

7.2.2 Unit Testing

Unit testing checks individual modules for correctness. Each component, such as face detection (Haar Cascade), eye detection, preprocessing, feature extraction, and classification, is tested separately to identify errors before integration.

7.2.3 Integration Testing

Integration testing evaluates how modules interact by passing outputs from one stage (e.g., face detection) as inputs to the next (eye detection). It ensures smooth data flow and resolves dependency or compatibility issues.

7.2.4 Validation Testing

Validation testing ensures the system meets user requirements. Real users test whether eye movements and blink inputs are accurately converted into commands, ensuring a reliable and user-friendly experience.

7.2.5 Functional testing

Functional testing verifies that all system functions operate correctly, including gaze detection, blink detection, command generation, and output execution. Each function is tested against expected results to ensure error-free performance.

7.3 TEST CASES

1. Face and Eye Detection Accuracy

Test Case: Use webcam streaming input and use with different users in varying lighting conditions.

Expected Outcome: The face should be detected by the system, as well as both eyes on it.

2. Eye State Detection (Open/Close)

Test Case: Eyes opens and closes (blink condition).

Expected Outcome: The system should detect eye state open or closed without delay.

3. Gaze Direction Detection

Test Case: Eye movements (left, right, center).

Expected Outcome: The system will classify the gaze direction real time (left, right, forward).

4. Command Mapping Accuracy

Test Case: Carry out eye commands in response (look left, right, blink).

Expected Outcome: The system should map the eye movements correctly to the commands like left, right, forward and stop.

5. Real-Time Wheelchair Control (3D Model)

Test Case: Plan to snip a species out of the frame and reinsert later.

Expected Outcome: the wheel-chair Model should move according to the eye direction (left → left, right → right, straight → forward blink stop).

6. System Response Time

Test Case: Delay between eye movement and system output.

Expected Outcome: Real-time response of the system (low latency - no lag).

Confusion Matrix (4-Class Eye Gaze Classification)

	Left	Right	Forward	Close
Left	TP	FP	FP	FP
Right	FP	TP	FP	FP
Forward	FP	FP	TP	FP
Close	FP	FP	FP	TP

Fig no:7.1 Confusion Matrix

Explanation:

- True Positive (TP): Correctly predicted eye movement class
- False Positive (FP): Incorrect prediction (wrong class assigned)
- False Negative (FN): Actual class not detected correctly
- True Negative (TN): Correct rejection of other classes

1) Accuracy:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Measures overall correctness of eye gaze classification (Left, Right, Forward, Close) in your system.

2) Precision:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Indicates how accurately the system predicts a specific eye movement (e.g., how many “LEFT LOOK” predictions are correct).

3) Recall:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Measures how well the system detects all actual eye movements (e.g., detects all true “RIGHT LOOK” cases).

4) F1-Score:

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Provides balanced performance between precision and recall for reliable eye gaze classification.

Class	Precision	Recall	F1-Score	Support
close_look	1.00	1.00	1.00	3828
forward_look	0.98	0.97	0.98	3457
left_look	0.99	0.99	0.99	3029
right_look	0.98	0.98	0.98	3577

Fig No.7.2: Classification Report

CHAPTER 8

RESULTS AND DISCUSSION

8.1 PERFORMANCE ANALYSIS

Performance analysis of the eye-based interaction system covering detection accuracy, classification reliability, real-time responsiveness and system robustness Face Detection module using Haar Cascade is tested for efficient detection of facial regions for various lighting conditions. Using MediaPipe for eye detection and landmark localization ensures accuracy for specific regions about the eyes, allowing tracking to remain stable even when head is turned slightly.

This involves analyzing the gaze direction and blink detection accuracy throughout to identify thresholds for classification of eye state (right, left, forward, open, close) while minimizing errors. We assess the prediction performance and inter-user consistency of the CNN-based classification model. Latency pendulum: When responding to a command, the system is challenged by the need to convert eye movements into commands as quickly as possible.

The performance in real-time is evaluated with respect to the frame processing speed (FPS), such that task runs seamlessly, even on regular hardware.

Results: 3D Wheelchair Control Module Command Accuracy -- Testing confirms that the correct movement matches eye input. The system achieves an accurate and robust performance with real-time response.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

The system integrates computer vision and machine learning to enable accurate, real-time eye-based interaction. Face detection, eye tracking, iris detection, blink detection, and CNN-based classification are successfully implemented, allowing reliable detection of eye movements and command generation. The system effectively controls the 3D wheelchair model, demonstrating smooth and responsive performance with minimal latency. Core modules, including document scrolling, are completed and functioning efficiently. The system supports gaze-based navigation and interaction. Overall, it provides a scalable, low-cost, and hands-free solution, enhancing accessibility and user independence in real-time applications.

9.2 FUTURE ENHANCEMENT

Future enhancements focus on converting the system into a hardware-based model using Raspberry Pi. By integrating a camera module and Python libraries like OpenCV, MediaPipe, and TensorFlow, real-time embedded processing can be achieved. Optimization techniques will reduce computational load and improve efficiency on low-power devices. Wireless modules can enable real-world control and monitoring. Improving model efficiency and reducing latency will further enhance performance. This integration will make the system portable, cost-effective, and suitable for real-time applications, improving usability and accessibility.

APPENDICES

APPENDIX 1

SOURCE CODE

←----Wheelchair-model----→

#Gaze_Tracking

```
import cv2
```

```
import mediapipe as mp
```

```
import numpy as np
```

```
from collections import deque
```

```
import socket
```

```
# ----- UDP Setup -----
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
SERVER_ADDRESS = ("127.0.0.1", 9999)
```

```
last_sent_command = None
```

```
# latency fix
```

```
# ----- MediaPipe Init -----
```

```
mp_face_mesh = mp.solutions.face_mesh
```

```
face_mesh = mp_face_mesh.FaceMesh(
```

```
    max_num_faces=1,
```

```
    refine_landmarks=True,
```

```
    min_detection_confidence=0.5,
```

```
    min_tracking_confidence=0.5)
```

```
# ----- Landmarks -----
```

```
LEFT_IRIS = [474, 475, 476, 477]
```

```
RIGHT_IRIS = [469, 470, 471, 472]
```

```
LEFT_EYE_CORNERS = [33, 133]
```



```

RIGHT_EYE_CORNERS = [362, 263]
LEFT_EYE_EAR = [33, 160, 158, 133, 153, 144]
RIGHT_EYE_EAR = [362, 385, 387, 263, 373, 380]
# ----- Parameters -----
EAR_THRESHOLD = 0.18
EYE_CLOSE_FRAMES = 5
SMOOTHING_FRAMES = 7
ratio_buffer = deque(maxlen=SMOOTHING_FRAMES)
close_counter = 0
# ----- EAR Function -----
def compute_ear(landmarks, eye_indices, w, h):
    pts = [(int(landmarks[i].x * w), int(landmarks[i].y * h)) for i in
eye_indices]
    v1 = np.linalg.norm(np.array(pts[1]) - np.array(pts[5]))
    v2 = np.linalg.norm(np.array(pts[2]) - np.array(pts[4]))
    h_dist = np.linalg.norm(np.array(pts[0]) - np.array(pts[3]))
    return (v1 + v2) / (2.0 * h_dist)
# ----- Webcam -----
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    h, w, _ = frame.shape
    results = face_mesh.process(rgb)
    gaze_text = "NO EYE"
    eye_closed = False

```

```

if results.multi_face_landmarks:
    for face_landmarks in results.multi_face_landmarks:
        # ----- Eye Close Detection -----
        left_ear = compute_ear(face_landmarks.landmark,
LEFT_EYE_EAR, w, h)
        right_ear = compute_ear(face_landmarks.landmark,
RIGHT_EYE_EAR, w, h)
        if left_ear < EAR_THRESHOLD and right_ear <
EAR_THRESHOLD:
            close_counter += 1
        else:
            close_counter = 0
        if close_counter >= EYE_CLOSE_FRAMES:
            gaze_text = "CLOSE LOOK"
            ratio_buffer.clear()
            eye_closed = True
            break
    ratios = [
# ----- LEFT EYE -----
left_iris = np.array([
    (int(face_landmarks.landmark[i].x * w),
    int(face_landmarks.landmark[i].y * h))
    for i in LEFT_IRIS])
    left_pupil = left_iris.mean(axis=0).astype(int)
    l_lc =
int(face_landmarks.landmark[LEFT_EYE_CORNERS[0]].x * w)
    l_rc =
int(face_landmarks.landmark[LEFT_EYE_CORNERS[1]].x * w)
    left_ratio = (left_pupil[0] - l_lc) / (l_rc - l_lc)

```

```

ratios.append(left_ratio)
cv2.circle(frame, tuple(left_pupil), 4, (0, 255, 0), -1)
# ----- RIGHT EYE -----
right_iris = np.array([
    (int(face_landmarks.landmark[i].x * w),
     int(face_landmarks.landmark[i].y * h))
    for i in RIGHT_IRIS ])
right_pupil = right_iris.mean(axis=0).astype(int)
r_lc =
int(face_landmarks.landmark[RIGHT_EYE_CORNERS[0]].x * w)
r_rc =
int(face_landmarks.landmark[RIGHT_EYE_CORNERS[1]].x * w)
right_ratio = (right_pupil[0] - r_lc) / (r_rc - r_lc)
ratios.append(right_ratio)
cv2.circle(frame, tuple(right_pupil), 4, (0, 255, 0), -1)
# ----- GAZE DECISION -----
if not eye_closed:
    avg_ratio = sum(ratios) / len(ratios)
    ratio_buffer.append(avg_ratio)
    smooth_ratio = sum(ratio_buffer) / len(ratio_buffer)
    if smooth_ratio < 0.42:
        gaze_text = "LEFT LOOK"
    elif smooth_ratio > 0.58:
        gaze_text = "RIGHT LOOK"
    else:
        gaze_text = "FORWARD LOOK"
# ----- SEND COMMAND (LOW LATENCY) -----
if gaze_text != last_sent_command:
    sock.sendto(gaze_text.encode(), SERVER_ADDRESS)

```

```

        last_sent_command = gaze_text
    cv2.putText(
        frame,
        gaze_text,(30, 45),
        cv2.FONT_HERSHEY_SIMPLEX, 1.2,
        (0, 255, 0), 3)
    cv2.imshow("High Efficiency Eye Gaze Tracking", frame)
    if cv2.waitKey(1) & 0xFF == 27:
        break
cap.release()
cv2.destroyAllWindows()

# Wheelchair_simulation
import pyglet
from pyglet.gl import *
from OpenGL.GLU import *
import trimesh
import numpy as np
import socket
import math

# ----- UDP Receiver -----
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("127.0.0.1", 9999))
sock.setblocking(False)
current_command = "STOP"

# ----- Load GLB Model -----
scene = trimesh.load("wheelchair.glb")
if isinstance(scene, trimesh.Scene):
    mesh = trimesh.util.concatenate(

```

```

        tuple(scene.geometry.values()))
else:
    mesh = scene
    # ----- Center & Normalize Model -----
    mesh.apply_translation(-mesh.centroid)
    scale = 1.0 / max(mesh.extents)
    mesh.apply_scale(scale * 2.0)
    vertices = np.array(mesh.vertices, dtype=np.float32)
    faces = np.array(mesh.faces, dtype=np.int32)
    # ----- Motion State -----
    pos_x, pos_z = 0.0, 0.0
    rotation_y = 0.0
    MOVE_SPEED = 0.05
    ROTATE_SPEED = 2.0
    # ----- Create Window -----
    window = pyglet.window.Window(
        1000, 700, "Eye-Gaze Controlled Wheelchair – Phase 2.3",
        resizable=True
    )
    glEnable(GL_DEPTH_TEST)
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glEnable(GL_COLOR_MATERIAL)
    # ----- Light Setup -----
    glLightfv(GL_LIGHT0, GL_POSITION, (GLfloat * 4)(3, 5, 5, 1))
    glLightfv(GL_LIGHT0, GL_DIFFUSE, (GLfloat * 4)(1, 1, 1, 1))
    # ----- Camera -----
    def setup_camera():
        glMatrixMode(GL_PROJECTION)
        glLoadIdentity()

```

```

gluPerspective(45, window.width / window.height, 0.1, 100)
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()
gluLookAt(
    pos_x, 3, pos_z + 6,
    pos_x, 0, pos_z,
    0, 1, 0)
# ----- Motion Update (LOW LATENCY) -----
def update_motion(dt):
    global pos_x, pos_z, rotation_y, current_command
    # FLUSH ALL OLD PACKETS – KEEP ONLY LATEST
    while True:
        try:
            data, _ = sock.recvfrom(1024)
            current_command = data.decode()
        except BlockingIOError:
            break
    # ----- Apply Motion -----
    if current_command == "LEFT LOOK":
        rotation_y += ROTATE_SPEED
    elif current_command == "RIGHT LOOK":
        rotation_y -= ROTATE_SPEED
    elif current_command == "FORWARD LOOK":
        rad = math.radians(rotation_y)
        pos_x -= MOVE_SPEED * math.sin(rad)
        pos_z += MOVE_SPEED * math.cos(rad)
    # CLOSE LOOK / NO EYE → STOP (do nothing)
# ----- Draw Ground -----
def draw_ground():

```

```

glDisable(GL_LIGHTING)
glColor3f(0.3, 0.3, 0.3)
glBegin(GL_QUADS)
glVertex3f(-10, -1, -10)
glVertex3f(-10, -1, 10)
glVertex3f(10, -1, 10)
glVertex3f(10, -1, -10)
glEnd()
glEnable(GL_LIGHTING)

# ----- Draw -----

@window.event
def on_draw():
    window.clear()
    setup_camera()
    draw_ground()
    glPushMatrix()
    glTranslatef(pos_x, 0, pos_z)
    glRotatef(rotation_y, 0, 1, 0)
    glColor3f(0.8, 0.8, 0.8)
    glBegin(GL_TRIANGLES)
    for face in faces:
        for idx in face:
            glVertex3f(*vertices[idx])
    glEnd()
    glPopMatrix()

# ----- Run (FASTER LOOP) -----

pyglet.clock.schedule_interval(update_motion, 1/120)
pyglet.app.run()

```

←----Documet-Scroll model---→

```
import cv2
import numpy as np
import pyautogui
from tensorflow.keras.models import load_model
# Load trained model
model = load_model("eye_model.h5")
# Start webcam
cap = cv2.VideoCapture(0)
print("Press 'q' to exit")
import time
# Variables (add before loop)
last_action_time = 0
cooldown = 1.0 # seconds
history = []
history_size = 5
while True:
    ret, frame = cap.read()
    if not ret:
        break
    # Flip frame for natural view
    frame = cv2.flip(frame, 1)
    # Resize and preprocess
    img = cv2.resize(frame, (64, 64))
    img = img / 255.0
    img = np.expand_dims(img, axis=0)
    # Prediction
```



```

pred = model.predict(img, verbose=0)[0][0]
# FIXED LOGIC
if pred >= 0.50:
    label = "UP"
elif pred <= 0.35:
    label = "DOWN"
else:
    label = "CENTER"
# Store history
history.append(label)
if len(history) > history_size:
    history.pop(0)
# Check stability
if history.count("UP") == history_size:
    stable_label = "UP"
elif history.count("DOWN") == history_size:
    stable_label = "DOWN"
else:
    stable_label = "CENTER"
# Cooldown check
current_time = time.time()
if current_time - last_action_time > cooldown:
    if stable_label == "UP":
        pyautogui.scroll(20)
        last_action_time = current_time
    elif stable_label == "DOWN":
        pyautogui.scroll(-20)
        last_action_time = current_time
# Display result

```

```

cv2.putText(frame, f'Movement: {label}', (20, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
cv2.imshow("Eye Movement Detection", frame)

# Exit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release() ; cv2.destroyAllWindows

```

APPENDIX 2

SCREENSHOTS OF WHEELCHAIR MOVEMENT

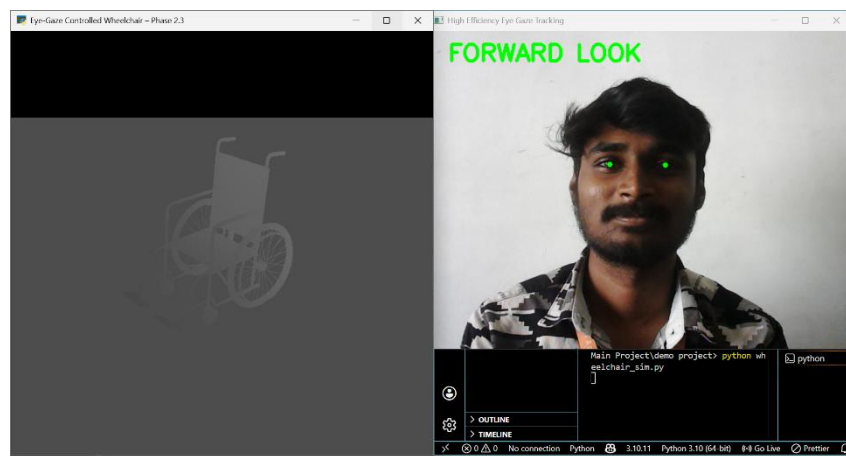


Fig No: 1 Forward Gaze Wheelchair Movement

The system detects a straight gaze as “FORWARD LOOK” and generates a forward movement command. When the user looks ahead, the 3D wheelchair moves straight, demonstrating accurate real-time control using eye gaze.

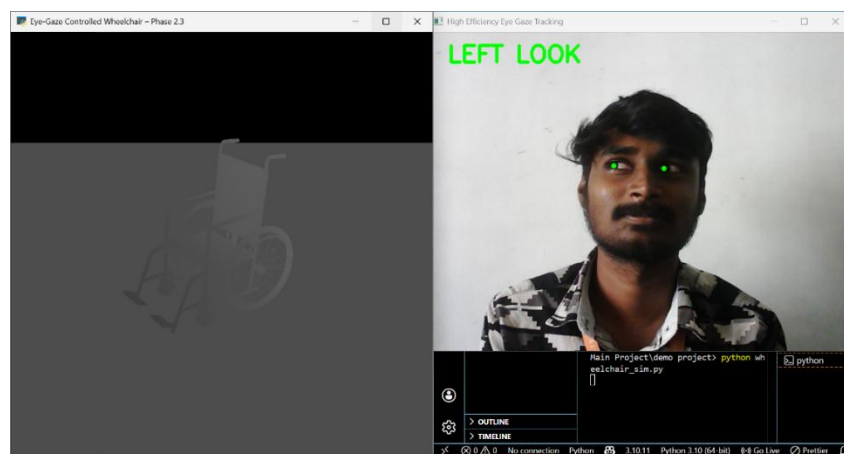


Fig No: 2 Left Gaze Wheelchair Control

The system detects a left gaze as “LEFT LOOK” and generates the corresponding command. When the user looks left, the 3D wheelchair turns left, confirming precise real-time mapping of gaze direction to movement.

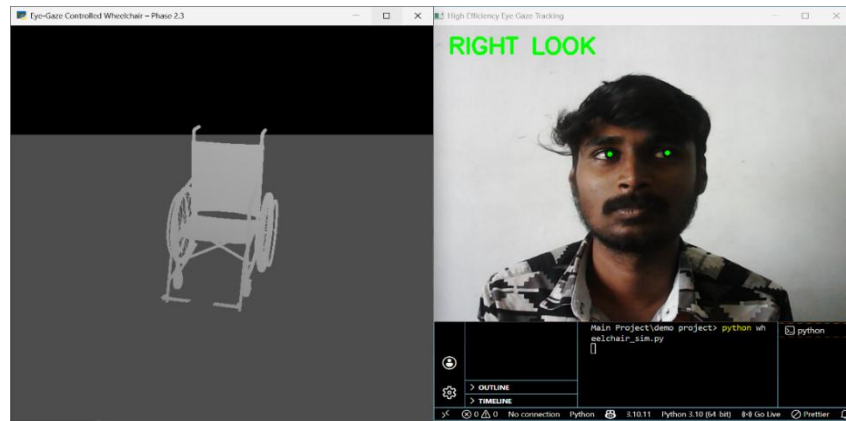


Fig No: 3 Right Gaze Wheelchair Control

The system recognizes when the user looks to the right and labels it as “RIGHT LOOK”. Based on this detection, it generates the appropriate control command. In this output, when the user looks right, the 3D wheelchair model also turns right. This confirms that eye gaze direction is accurately linked to wheelchair movement in real time.

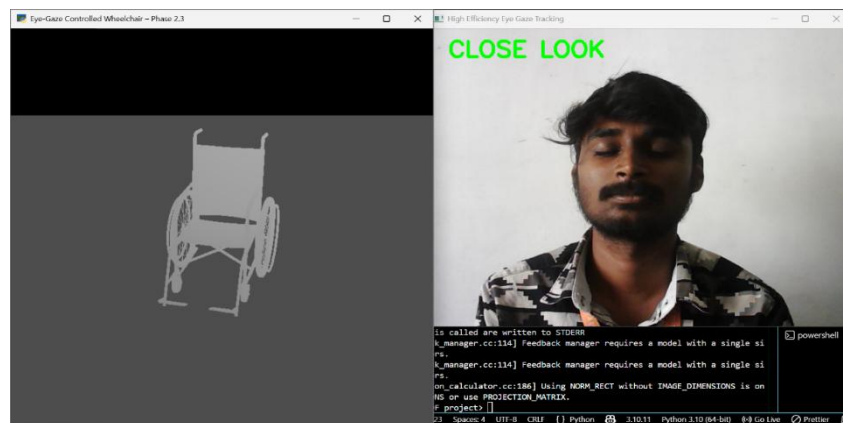


Fig No: 4 Eye Blink Wheelchair Stop

The system detects eye closure and classifies it as “CLOSE LOOK,” which acts as a stop command for the wheelchair. When the user

closes their eyes, the system immediately halts movement, ensuring safety and providing an intuitive emergency control.

SCREENSHOTS OF DOCUMENT SCROLLING

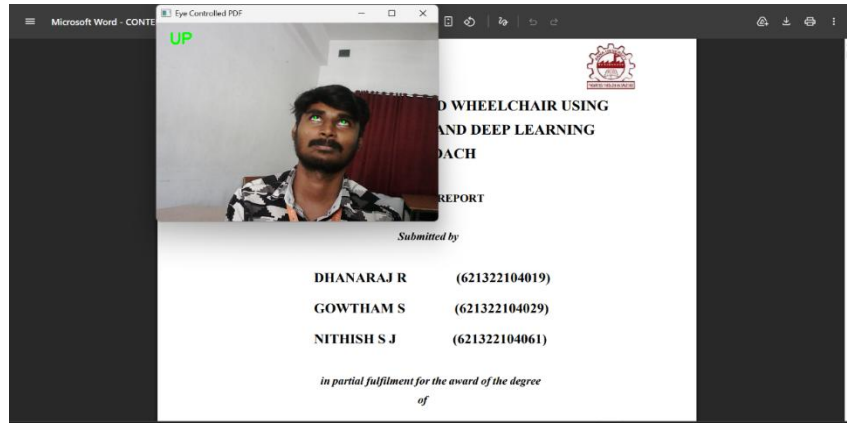


Fig No: 5 Upward Gaze Document Scroll

The system detects upward eye movement and classifies it as “UP LOOK.” This is mapped to a document control command, causing the PDF to scroll upward. It demonstrates effective hands-free document navigation.

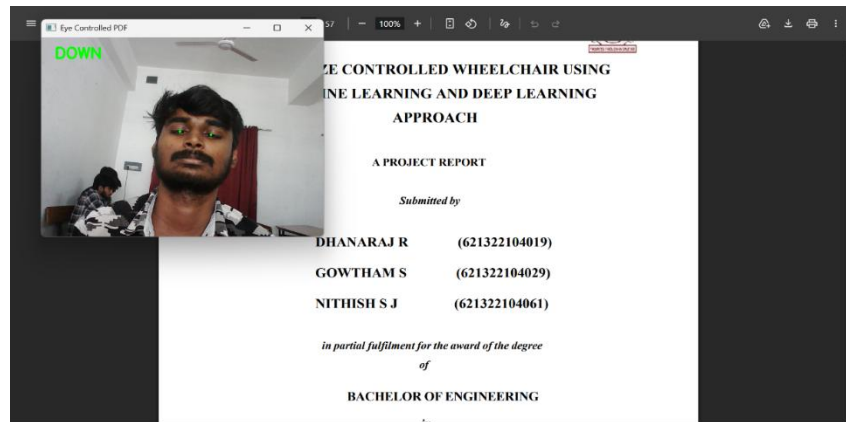


Fig No: 6 Downward Gaze Document Scroll

The system detects downward eye movement as “DOWN LOOK,” triggering downward PDF scrolling. This enables smooth and hands-free document control.

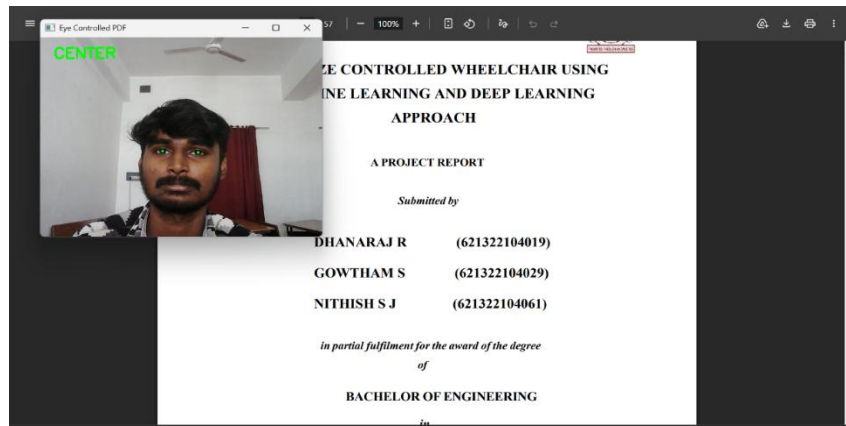


Fig No: 7 Center Gaze Document Pause

The system detects a straight gaze as “CENTER LOOK,” which stops document scrolling. This ensures precise control and stability for efficient hands-free reading.

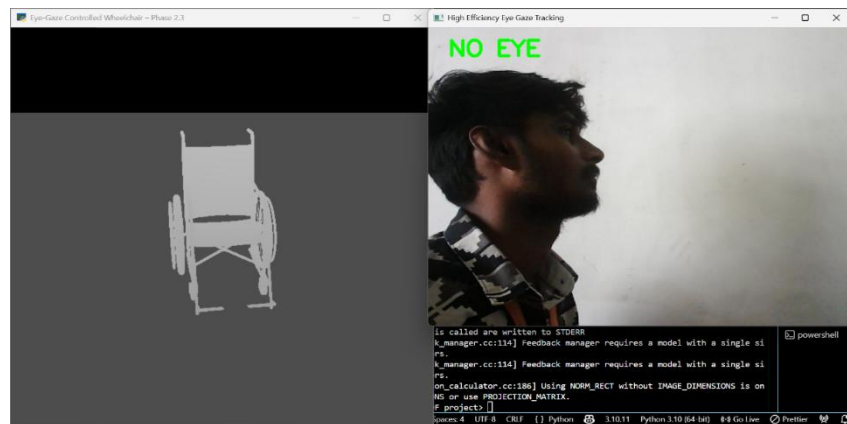


Fig No: 8 No-Eye Direction Wheelchair Movement

The system detects the absence of visible eyes and classifies it as “NO EYE”. This condition occurs when the user turns away or the face is not properly detected. In this case, the system automatically stops the wheelchair to ensure safety. This prevents unintended movements and enhances system reliability during real-time operation.

REFERENCES

1. P. Kielty, C. Ryan, W. Shariff, J. Lemley, and P. Corcoran, “Event-Based Multi-Task Facial Landmark and Blink Detection,” *IEEE Access*, vol. 13, pp. 45609–45618, Mar. 2025.
2. Chaki T, Kawakami T, Tanguy A, Yoshiike T and Kheddar A, “Humanoid-human sit-to-stand-to-sit assistance,” *IEEE Robotics and Automation Letters*, vol. 10, pp. 210–218, 2025.
3. Wang C, Wang R, Leng Y, Iramina K, Yang Y and Ge S, “An eye movement classification method based on cascade forest,” *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 12, pp. 7184–7193, Dec. 2024.
4. Li N, Chang M and Raychowdhury A, “E-Gaze: Gaze estimation with event camera,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, pp. 1450–1460, 2024.
5. Zhang Y, Li Q, Nahata S, Jamal T, Cheng S K, Cauwenberghs G and Jung T P, “Integrating large language model, EEG, and eye-tracking for word-level neural state classification in reading comprehension,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 32, pp. 3465–3475, 2024.
6. Artiran S, Bedmutha P S and Cosman P, “Analysis of gaze, head orientation, and joint attention in autism with triadic VR interviews,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 32, pp. 759–769, 2024.

7. Masud U, Almolhis N A, Alhazmi A, Ramakrishnan J, Islam F U and Farooqi A R, “Smart wheelchair controlled through a vision-based autonomous system,” *IEEE Access*, vol. 12, pp. 65099–65118, 2024.
8. Hassan M A, Aldridge C M, Zhuang Y, Yin X, McMurry T, Rohde G K and Southerland A M, and A. M. Southerland, “Approach to quantify eye movements to augment stroke diagnosis with a non-calibrated eye-tracker,” *IEEE Transactions on Biomedical Engineering*, vol. 70, no. 6, pp. 1750–1757, June 2023.
9. M. Abul Hassan, C. M. Aldridge, Y. Zhuang, et al., “Approach to quantify eye movements to augment stroke diagnosis with a non-calibrated eye-tracker,” *IEEE Journal of Biomedical and Health Informatics*, vol. 27, pp. 345–353, 2023.
10. Edughele H O, Zhang Y, Muhammad-Sukki F, Vien Q T, Morris-Cafiero H and Agyeman M O, “Eye-tracking assistive technologies for individuals with amyotrophic lateral sclerosis,” *IEEE Access*, vol. 10, pp. 41952–41963, 2022.
11. Wang X, Zhang J, Zhang H, Zhao S and Liu H, “Vision-Based Gaze Estimation: A Review,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 14, no. 2, pp. 316–326, June 2022.
12. Kang D and Ma L, “Real-Time Eye Tracking for Bare and Sunglasses-Wearing Faces for Augmented Reality 3D Head-Up Displays,” *IEEE Access*, vol. 9, pp. 125508–125518, 2021.
13. Sun X and Balasingam B, “Reading line classification using eye-trackers,” *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–10, 2021.
14. Akinyelu A and Blignaut P, “Convolutional Neural Network-Based Methods for Eye Gaze Estimation: A Survey,” *IEEE Access*, vol. 8, pp. 142581–142592, 2020.

15. Ali A and Kim Y G, “Deep Fusion for 3D Gaze Estimation From Natural Face Images Using Multi-Stream CNNs,” *IEEE Access*, vol. 8, pp. 69212–69220, Apr. 2020.
16. Sugano Y, Fritz M and Bulling A, “MPIIGaze: Real-world dataset and deep appearance-based gaze estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 1, pp. 162–175, Jan. 2019.
17. Lian D, Hu L, Luo W, Xu Y, Duan L, Yu J and Gao S, “Multiview multitask gaze estimation with deep convolutional neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 10, pp. 3010–3021, Oct. 2019.
18. Kononenko D, Ganin Y, Sungatullina D and Lempitsky V, “Photorealistic monocular gaze redirection using machine learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 11, pp. 2696–2704, Nov. 2018.
19. Cheung Y M and Peng Q, “Eye gaze tracking with a web camera in a desktop environment,” *IEEE Transactions on Human-Machine Systems*, vol. 45, no. 4, pp. 419–430, Aug. 2015.
20. Vicente F, Huang Z, Xiong X, De la Torre F, Zhang W and Levi D, “Driver gaze tracking and eyes off the road detection system,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 4, pp. 2014–2023, Aug. 2015.
21. Al-Rahayfeh A and Faezipour M, “Eye Tracking and Head Movement Detection: A State-of-Art Survey,” *IEEE Journal of Translational Engineering in Health and Medicine*, vol. 1, pp. 2100212, Nov. 2013.
22. Kim C, Choi S I, Turk M and Choi C H, “A new biased discriminant analysis using composite vectors for eye detection,” *IEEE Transactions on Systems, Man, and Cybernetics—Part B: Cybernetics*, vol. 42, no. 4, pp. 1095–1104, Aug. 2012.

23. Hansen D W and Ji Q, “In the eye of the beholder: A survey of models for eyes and gaze,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 32, no. 3, pp. 478–500, Mar. 2010.
24. Perez C A, Lazcano V A and Estevez P A, “Real-time iris detection on coronal-axis-rotated faces,” IEEE Transactions on Systems, Man, and Cybernetics—Part C: Applications and Reviews, vol. 37, no. 5, pp. 971–978, Sept. 2007.
25. Zhu Z and Ji Q, “Novel eye gaze tracking techniques under natural head movement,” IEEE Transactions on Biomedical Engineering, vol. 54, no. 12, pp. 2246–2256, Dec. 2007.

PUBLICATION

Dr. A. RAJAPRABU DHANARAJ R (621322104019), GOWTHAM S (621322104029) AND NITHISH S J (621322104061) has published a paper entitled **“EYE-GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING APPROACH”** in Proceedings of an International Conference, pp. 554–563, 2026.



INTERNATIONAL CONFERENCE ON INNOVATION & RESEARCH IN MULTIDISCIPLINARY DOMAINS
(ICIRMD-2026)

CERTIFICATE OF PARTICIPATION

This is certified that Dr./Mr./Ms.

Dr. A. RAJAPRABU

who has successfully presented the paper entitled

**EYE- GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING
APPROACH**

at **International Conference on Innovation & Research in Multidisciplinary Domains (ICIRMD-2026)**
held on 31st January 2026

C. Jagan 21/1

Conference Co-Chair

Dr. A. Rajaprabu

Conference Chair

Dr. A. Rajaprabu

General Chair



INTERNATIONAL CONFERENCE ON INNOVATION & RESEARCH IN MULTIDISCIPLINARY DOMAINS
(ICIRMD-2026)

CERTIFICATE OF PARTICIPATION

This is certified that Dr./Mr./Ms.

DHANARAJ R

who has successfully presented the paper entitled

**EYE- GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING
APPROACH**

at **International Conference on Innovation & Research in Multidisciplinary Domains (ICIRMD-2026)**
held on 31st January 2026

C. Jagan

Conference Co-Chair

Dr. Jagan

Conference Chair

Dr. Jagan

General Chair



INTERNATIONAL CONFERENCE ON INNOVATION & RESEARCH IN MULTIDISCIPLINARY DOMAINS
(ICIRMD-2026)

CERTIFICATE OF PARTICIPATION

This is certified that Dr./Mr./Ms.

GOWTHAM S

who has successfully presented the paper entitled

**EYE- GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING
APPROACH**

at **International Conference on Innovation & Research in Multidisciplinary Domains (ICIRMD-2026)**
held on 31st January 2026

Conference Co-Chair

Conference Chair

General Chair



INTERNATIONAL CONFERENCE ON INNOVATION & RESEARCH IN MULTIDISCIPLINARY DOMAINS
(ICIRMD-2026)

CERTIFICATE OF PARTICIPATION

This is certified that Dr./Mr./Ms.

NITHISH S J

who has successfully presented the paper entitled

**EYE- GAZE CONTROLLED WHEELCHAIR USING MACHINE LEARNING AND DEEP LEARNING
APPROACH**

at **International Conference on Innovation & Research in Multidisciplinary Domains (ICIRMD-2026)**
held on 31st January 2026

Conference Co-Chair

Conference Chair

General Chair